



TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Fakultät Elektrotechnik Feinwerktechnik Informatik

Conceptual design and realization of a graphical tool for determining the test coverage of digital circuit diagrams

Masterarbeit im Studiengang Softwareengineering und
Informationstechnik

vorgelegt von

Razvan Matis

Matrikelnummer 2403005

Erstgutachter: Prof. Dr. Flaviu Popp-Nowak

Zweitgutachter: Prof. Dr. Claus Kuntzsch

© 2021

Dieses Werk einschließlic seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung auSSerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

According to Printed circuit board (PCB)s and their digital circuits which are getting more complex all the time there is the requirement of being able to determine the test coverage of it. In dependency on these determined values then automatic boundary scan chains could be performed or the need of changing something on the layout will be visible. This document describes the way of the conceptual design and the realization of a graphical tool for loading, showing different digital circuit diagrams projects which need to be in the Open database (ODB)++ format and the determination of test coverage parameters as well as the creation of Serial vector format (SVF) files for being able to run the boundary scan chain on these special devices which contains at least one Joint test action group (JTAG) device. You will get detailed information about the frameworks, programming patterns and best practices which were used for creating this Model-view-viewModel (MVVM) C# project and the project structure itself which was created for making sure to have the following features available:

- Opening any ODB++ project by connecting to a stand alone Google remote procedure calls (GRPC) server and using the PCB-Investigator API
- Exporting any opened projects to Java script object notation (JSON) file for later usage and importing of that created files
- Creation and loading of project files which contains a manifest file for information about the project structure itself
- Modifying, exporting and importing of different kind of settings according the connection to the GRPC server, the appearance and the test coverage values
- Determination of different test coverage values according to the boundary scan mechanism
- Creation of SVF files for run the boundary scan chain with a programmer from ProMik on a JTAG containing device

Contents

Abstract	ii
1. List of abbreviations	1
2. Introduction	2
2.1. Motivation	2
2.2. Requirements	2
2.3. Programming language and operating system	4
3. Fundamentals	5
3.1. PCB-Investigator	5
3.2. ODB++ project format	7
3.3. JTAG interface	7
3.4. Boundary scan	8
3.5. SVF file format	9
3.6. Model-view-viewModel	9
3.7. GRPC	9
3.8. JSON format	9
4. Conceptual design	10
5. Architecture	11
5.1. Overview	12
5.2. Interfaces	12
5.3. Grpc Server for using the PCB Investigator API	12
5.4. Grpc Client for connecting to the Server and parsing all objects	12
5.5. GUI	12
5.5.1. Events	12
5.5.2. Helper	12
5.5.3. Interfaces	12
5.5.4. Implementations	12
5.5.5. Model	12
5.5.6. ViewModels	12
5.5.7. Views	12
5.5.8. MainView and MainViewModel	12

5.6. Test coverage determination of loaded project	12
5.7. Pin information extraction	12
5.8. SVF file generator	12
5.9. Unit tests	12
6. Analysis	13
6.1. Performance of loading ODB++ projects by using the PCBInvestigator API	13
6.2. Performance of loading projects by importing JSON formatted files	13
6.3. Performance of exporting projects into JSON format	13
6.4. Performance of showing connections of components	13
6.5. Performance of determination of test coverage values	13
7. Conclusion	14
8. Thanksgiving	15
A. Appendix	17
List of Figures	18
List of Tables	19
List of Listings	20
Bibliography	21

Chapter 1.

List of abbreviations

PCB	Printed circuit board
ODB	Open database
JTAG	Joint test action group
SVF	Serial vector format
GRPC	Google remote procedure calls
JSON	Java script object notation
MVVM	Model-view-viewModel
MCU	Micro controller
GUI	Graphical user interface
CPU	Central processing unit
RAM	Random access memory
GB	Gigabyte
CAD	Computer-aided design
aka	also known as
EXE	executable
API	Application interface

Chapter 2.

Introduction

Within this chapter i will describe the motivation, the requirements, the used programming language and operating system for creation of the whole project. All these aspects where previously discussed with the company to make sure to create a most valuable software for them.

2.1. Motivation

As the company ProMik GmbH is already providing programmers for being able to write date into different Micro controller (**MCU**)s now there came up the requirement, in dependency on customer demands, to perform boundary scan chains on **JTAG** devices. For being able to perform these kind of tests there should be a software available which allows to determine first the test coverage parameters in dependency on the boundary scan mechanism and create such **SVF** files for playing them with the programmer. This software should be provided as a complex framework where the opportunity is given to extend it in the future in elementary way by using much interfaces within the **MVVM** pattern.

2.2. Requirements

In order to provide all features as needed there were a couple of requirements which popped up at the very beginning of the conceptual design and during the implementation of the software:

- The software itself should consist of a Graphical user interface (**GUI**) application and a separate **GRPC** server for using the **PCB**-Investigator API
- There should be the possibility of opening **ODB++** project folders and display the result of circuit diagram in a useful way

- There should be the possibility of moving the visible area by mouse movement and zooming in and out of the visible area
- The visibility of each component should be settable
- All connections of one component to the others should be displayable
- The whole view of all components could be mirrorable along both axes
- The net names should be settable to visible and hidden
- An export mechanism of an opened project into **JSON** file for later usage should be available as an according import mechanism to be independent from the **PCB-Investigator** API as this requires a separate license for usage
- Settings should be available, exportable and importable
 - General settings according the appearance of the loaded project view
 - **GRPC**-Server related connection settings
 - General settings according the request of **PCB-Investigator** API
 - Test coverage related settings for determining all components for the boundary scan chain
 - **SVF** related settings according the programmer connection parameters to use
- Determination of test coverage parameters using the boundary scan mechanism
- Export of a file containing all pin information of all **JTAG** devices
- Creation of **SVF** files containing all instructions to be performed by the programmer while playing the boundary scan chain on a **JTAG** compatible device
- In order to create an extended-friendly framework there was furthermore the need of usage of company internal libraries instead of creating complete new ones
 - The usage of log panel was required
 - The usage of setting panel was required
 - The usage of **SVF** player was required

2.3. Programming language and operating system

The whole project was implemented with the software development environment called **Microsoft Visual Studio 2019** and the programming language **C#**. This is a quit powerful and modern language for creation of different desktop applications. The developer environment is really comfortable too, as it provides a NuGet package manager for organizing all external included libraries. Furthermore the whole project was created on a operating system **Windows 10 Professional x64** using a Central processing unit (**CPU**) of type **Intel(R)Xeon(R) E3-1240 v6 @ 3.70 GHz** and a total amount of Random access memory (**RAM**) of **16 Gigabyte (GB)**. This system makes sure to be able to implement without any delays of waiting for background processes at all and ensures to be able to run the developed software almost immediately without any delay times.

Chapter 3.

Fundamentals

3.1. PCB-Investigator

The **PCB**-Investigator is a software provided by EasyLogix also known as (**aka**) Schindler & Schill GmbH and provides many functionality regarding the opening, viewing and measuring of different kind of Computer-aided design (**CAD**) project files[1]:

- **ODB++**
- Gerber 274x
- Gen Cad 1.4
- Eagle files
- IDF files
- DXF files
- DPF files
- IPC2581 files
- IPC356 files
- PDF

Further the executable (**EXE**) file application itself provides different public functions which are accessible by adding the **PCB-Investigator.exe** as a reference to the project. As the company is just interested in extracting all relevant information from any **ODB++** relative projects the following public functions where used by the software[2]:

- `IAutomation.IAutomationInit()` - Initialization of the Application interface (**API**)
- `IAutomation.CreateNewPCBIWindow(bool visible)` - Creates the base window
- `IPCBIWindow.LoadData(string path)` - Loads a specific project into the window

- IPCBIWindow.GetStepList() - Getter for all steps of the window
- IStep.GetNets() - Getter for all nets of a step
- IStep.GetAllLayerNames() - Getter for all layer names of a step
- IStep.GetLayer(string layerName) - Getter for a layer in dependency on the layer name
- ILayer.GetAllLayerObjects() - Getter for all objects which are located on a specific layer

After the functions to be used are identified then the structure itself of the received objects need to be analyzed more in detail:

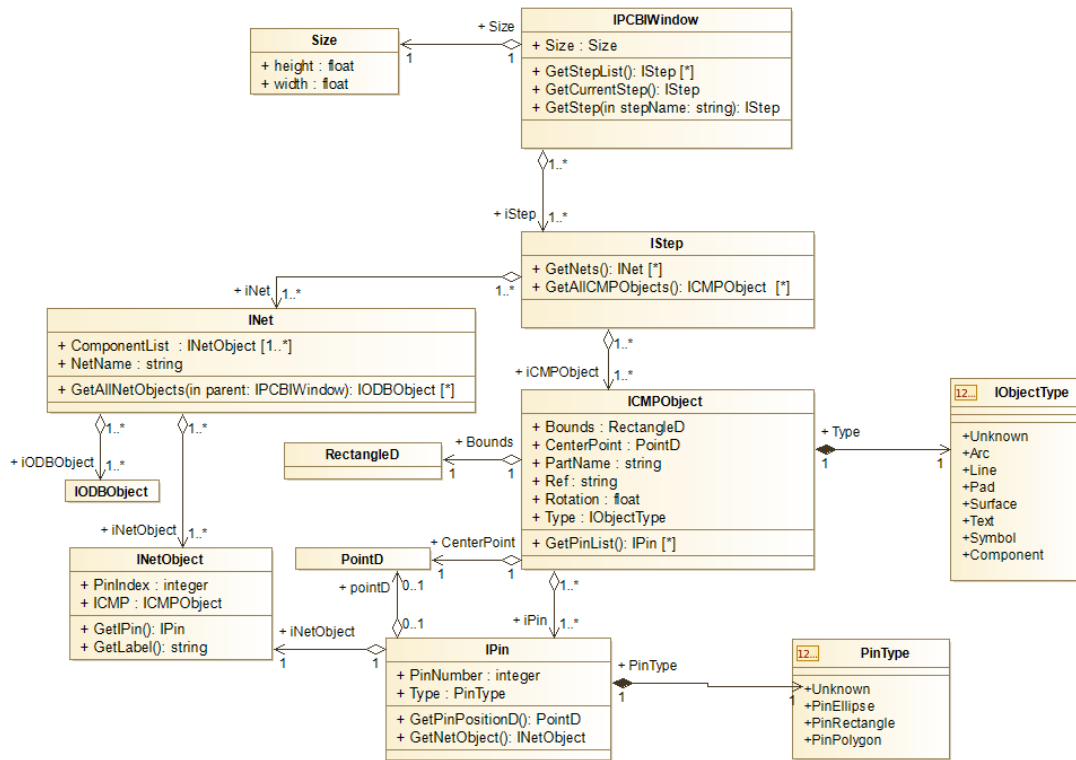


Figure 3.1.: Class diagram of the PCB-Investigator objects of interest

As shown within 3.1 you can see that the very first class is the IPCBIWindow which contains a various amount of IStep objects. Within this IStep objects there are many INet objects and ICMPObjets.

3.2. ODB++ project format

The ODB++ format is a de facto standard format for creation of CAD related projects[3] within the circuit board industry. It hierarchically contains all necessary information about the circuit diagram in detail:

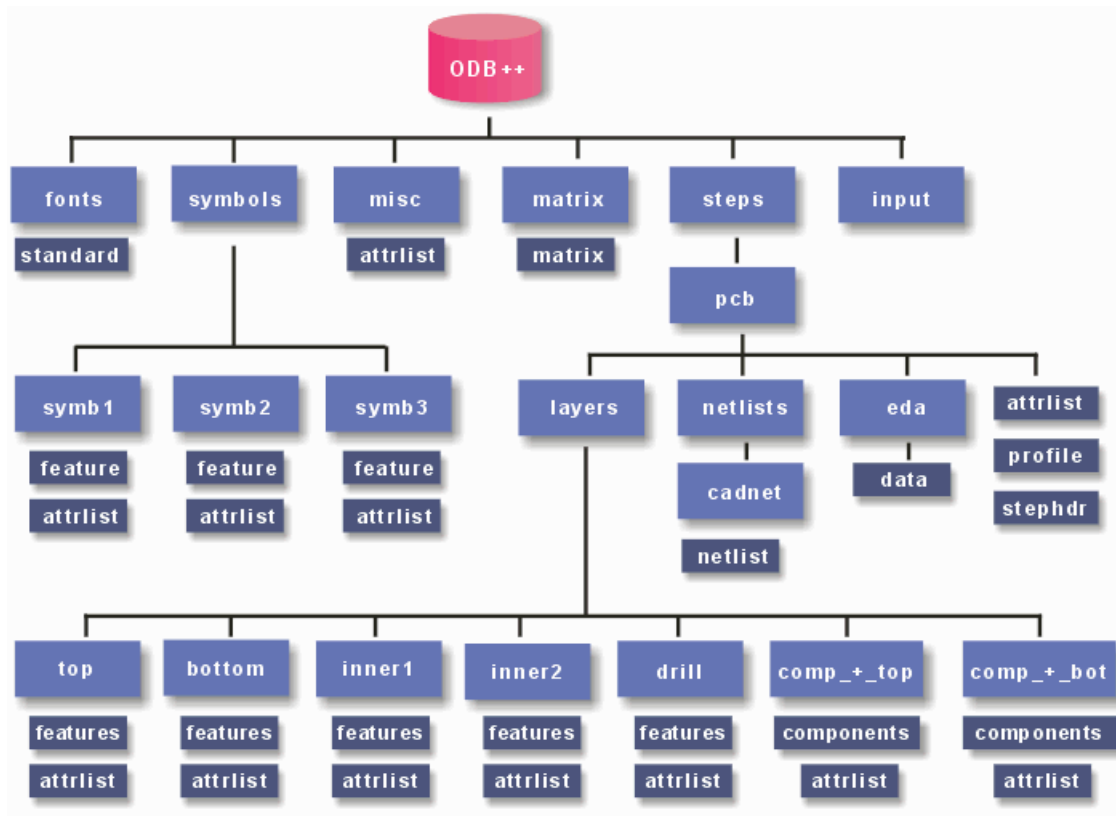


Figure 3.2.: ODB++ format content[4]

As shown in 3.2 you can see that this format contains all necessary components, net lists and attribute lists which are necessary for later determination of the test coverage of the circuit diagrams. These data will give detailed information regarding the placed components, their names, positions, sizes and connection to all nets. In dependency on the net connection the connected components could then be determined

3.3. JTAG interface

The JTAG interface is a debug related serial interface consisting of the following components in general[5]:

- Pin TDI - data in

- Pin TDO - data out
- Pin TMS - control
- Pin TCK - clock
- Pin TRST - test reset
- Boundary scan registers
- Boundary scan cells
- External connection pins
- Core logic
- Debug and programming registers

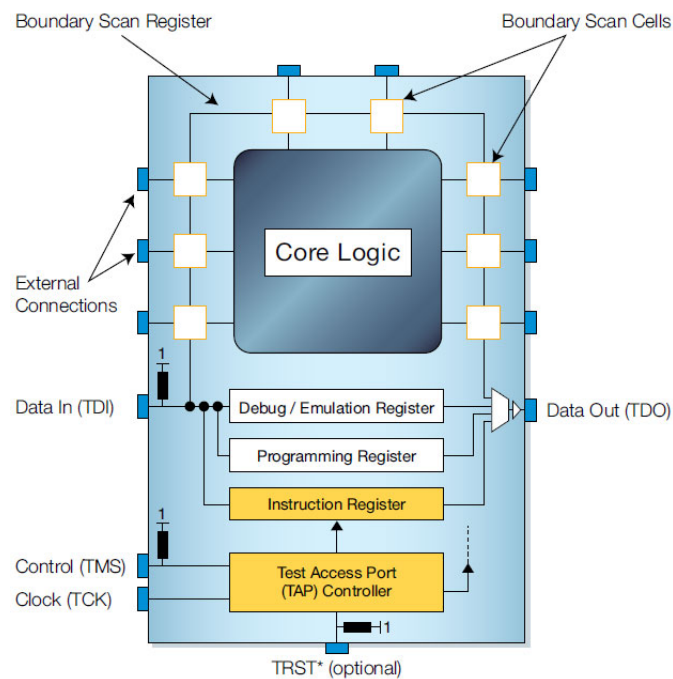


Figure 3.3.: JTAG interface in detail

As shown in 3.3 you can see that that the **JTAG** device itself already contains some addition core logic for being able to perform different kind of test-debugging functions.

3.4. Boundary scan

The main idea of the boundary scan is the purpose of ensuring that every component is mounted correctly in physically way as described within the circuit diagram[6]. In detail

that means that according to each external pin connection you are in general able to set any value to high or low and read back the physically present value of wiring. So if you know for example that on a specific pin position an according pull down resistor is being connected then you further know that in every case the read back value should be low. Additionally you can assume that if a pull up resistor is being connected to a pin independently on the set value the read back value should be high. Finally you can accept the fact that if some other components are connected to the pin which are not belonging to the pull down nor the pull up resistors then the read back value should be that one which was previously set. To perform a complete test of a device you must follow the steps:

1. The clock rate must be triggered as long as the complete boundary scan register length totally is[7]
2. During that time the TDI data will be forwarded into each boundary scan cell
3. If all cells are covered by a value then the TMS pin should be triggered for passing all read values into the connected pins
4. The clock rate must be triggered again for the total length of the boundary scan register
5. During that time all values from the external pins are being read out and transmitted into the TDO
6. Now the read out values could be compared with some expected values to detect differences

3.5. SVF file format

3.6. Model-view-viewModel

3.7. GRPC

3.8. JSON format

Chapter 4.

Conceptual design

Chapter 5.

Architecture

5.1. Overview

5.2. Interfaces

5.3. Grpc Server for using the PCB Investigator API

5.4. Grpc Client for connecting to the Server and parsing all objects

5.5. GUI

5.5.1. Events

5.5.2. Helper

5.5.3. Interfaces

5.5.4. Implementations

5.5.5. Model

5.5.6. ViewModels

5.5.7. Views

5.5.8. MainView and MainViewModel

5.6. Test coverage determination of loaded project

5.7. Pin information extraction

5.8. SVF file generator

5.9. Unit tests

Chapter 6.

Analysis

- 6.1. Performance of loading ODB++ projects by using the PCBInvestigator API
- 6.2. Performance of loading projects by importing JSON formatted files
- 6.3. Performance of exporting projects into JSON format
- 6.4. Performance of showing connections of components
- 6.5. Performance of determination of test coverage values

Chapter 7.

Conclusion

Chapter 8.

Thanksgiving

Within this small chapter I would like to thank a couple of people who made this work possible by supporting me in many different areas of my life.

First of all I want to thank my parents **Gabriela** and **Nicolae Matis** who have born me and supported me in every single minute of my life. I know I unfortunately didn't always made you proud of my actions but I hope that I am still working on it successfully by milestones like this one. Mum I really hope for you that you will solve your physical constraints successfully and dad I hope that you stay as young as you are for a really long time

Afterward I would like to thank my brother **Nik Matis** and my sister **Michelle Matis** who have supported me in that way as a brother just could expect it to be perfect. Michelle I wish you all the best with your study and Nik I hope you will find your destiny in choosing a new job soon.

And finally I would like to thank a couple of my best friends who have supported me in investing much free time with myself and were successfully responsible for being able to forget all the stress about the work and enjoy my life.

Andreas Korkenländer I already know you now about for 20 years and really hope that this will continue until my death. I wish you all the best with your new job and your intention of traveling around the world.

Dominik Müller you were my first best friend after I moved to Nürnberg and I really hope that connection persists for a longer time although you are sometimes really a hardheaded person :) I wish you all the best with your new challenge of doing a new apprenticeship.

Rene Biedermann you are one of that guys I'm able to speak about philosophic and esoteric topics and of course medical topics about my own health in a detailed way as I'm really impressed of it. I wish you all the best in accomplishing your exam for being

an alternative practitioner and I really think that this will satisfy your life in complete way.

Willi Fritz I would really like to thank you especially for the last year where you permitted me to do my training within your own flat and visited me almost every weekend for just hanging around and playing PlayStation 4 or for cooking some real tasty meals. I wish you all the best at finishing your tax consultant state examination and am really looking forward of traveling around the world with your person soon.

Appendix A.

Appendix

List of Figures

3.1. Class diagram of the PCB-Investigator objects of interest	6
3.2. ODB++ format content[4]	7
3.3. JTAG interface in detail	8

List of Tables

List of Listings

Bibliography

- [1] “Pcb-investigator,” 4/7/2021. [Online]. Available: <https://www.pcb-investigator.com/de/functions>
- [2] “Pcb-investigator interface documentation - table of content,” 4/7/2021. [Online]. Available: <https://manual.pcb-investigator.com/InterfaceDocumentation/index.php>
- [3] ODB++Design, “Why odb++design - odb++design,” 5/18/2020. [Online]. Available: <https://odbplusplus.com/design/why-odb-d/>
- [4] “What is odb++,” 2/5/2019. [Online]. Available: https://www.artwork.com/odb++/odb++_overview.htm
- [5] XJTAG, “Was ist jtag und wie kann ich es nutzen? - xjtag tutorial,” 8/27/2020. [Online]. Available: <https://www.xjtag.com/de/about-jtag/what-is-jtag/>
- [6] Harry Bleeker, *Boundary-Scan Test: A Practical Approach*. Springer Science+Business Media Dordrecht, 1993.
- [7] R.C. Cofer and Benjamin F. Harding, *Rapid System Prototyping with FPGAs*. Elsevier Inc., 2006.